Milestone-2

INVENTRA–INTELLIGENT INVENTORY MANAGEMENT SYSTEM

MODULE 2: PRODUCT / INVENTORY MANAGEMENT

- DEVELOPED USING: JAVA 17+, SPRING BOOT, SPRING SECURITY
- DATABASE: MYSQL
- FRONTEND: HTML, CSS, BOOTSTRAP, THYMELEAF

Presented by: Arya Desavale

Problem Statement

Businesses face challenges in managing product inventory efficiently.

Without a proper product module:

- Manual stock updates cause errors
- No real-time stock tracking
- Duplicate product entries
- No SKU uniqueness control
- No stock validation before selling
- Poor supplier & category validation
- Risk of stockouts and overstocking

There is a need for :

- Structured product management
- Automated stock updates
- Validation-driven operations
- Real-time inventory accuracy

EXISTING PROBLEM

Traditional inventory systems:

❌ Allow duplicate SKU entries
❌ No stock validation before stock-out
❌ No minimum stock monitoring
❌ No transaction logging
❌ No product status control (ACTIVE / INACTIVE)

These systems lead to:

- Inconsistent inventory data
- Revenue loss due to stock errors
- Manual correction effort
- Lack of transparency

PROPOSED SYSTEM:

Inventra Authentication Module provides:

- ✅ Add product with validation
- ✅ Unique SKU enforcement
- ✅ Stock In / Stock Out functionality
- ✅ Minimum stock level tracking
- ✅ Transaction logging
- ✅ Low stock alert mechanism
- ✅ Active / Inactive product control
- ✅ Layered architecture implementation

Security implemented using:

- Spring Security
- JWT Utility
- Encrypted password hashing
- Email reset token mechanism

PROPOSED SYSTEM:

Module Architecture

Controller Layer

- ProductController
- Handles HTTP requests
- No business logic

Service Layer

- ProductService
- Implements business logic

Validation Layer

- ProductValidationService
- Enforces business rules

Repository Layer

- ProductRepository
- Handles database operations

Support Services

- TransactionService
- AlertService

1. This separation improves:
2. Maintainability
3. Scalability
4. Clean Code structure

CLASSES IN MODULE 2

The module consists of:

- Product (Entity)
- ProductController
- ProductService
- ProductValidationService
- ProductRepository
- TransactionService
- AlertService

Product Entity

Product Class

Fields:

- productId (Primary Key)
- sku (Unique)
- name
- category
- supplier
- unitPrice
- stockQuantity
- minStockLevel
- status (ACTIVE / INACTIVE)

Purpose:

- Stores product information
- Maintains stock data
- Tracks product state
- Database Table: PRODUCTS

Validation Rules (Add Product)

Validation for Add Product

1. SKU must be unique
2. Product name must not be empty
3. Unit price must be > 0
4. Category must be valid
5. Supplier must be registered
6. Minimum stock level ⩾ 0
7. Initial stock ⩾ 0

Why validation is important?
Prevents duplicate entries
Avoids pricing errors
Maintains business consistency

Validation Rules (Stock In & Stock Out)

🔍 Stock In Validation
Product must exist
Product must be ACTIVE
Quantity must be > 0

🔍 Stock Out Validation
Product must exist
Product must be ACTIVE
Quantity must be > 0
Stock must be sufficient

This prevents:
Selling inactive products
Negative stock
Invalid transactions

ProductController (Thin Controller)

Responsibilities:

- Receives API requests
- Delegates to ProductService
- No validation logic
- No database logic

Functions:

addProduct()
stockIn()
stockOut()
✔ Keeps controller clean
✔ Follows Single Responsibility Principle

ProductService (Business Logic Layer)

Core responsibilities:

- Coordinates operations
- Calls validation service
- Updates stock
- Saves to database
- Logs transactions
- Checks low stock

Key Functions:

- addProduct()
- stockIn()
- stockOut()

ProductValidationService

Handles all business validations:

- validateNewProduct()
- validateStockIn()
- validateStockOut()

Returns:

- "VALID"

OR

- Specific error message

Benefits:

- Centralized validation logic
- Reusable
- Easy to modify business rules

TransactionService

Purpose:
Logs every stock operation

Logs:

- STOCK_IN
- STOCK_OUT
- Product ID
- Quantity
- Timestamp

Ensures:

- Audit trail
- Transparency
- Accountability

Pseudocode for Each Class

CLASS ProductController

```
FUNCTION addProduct(request)
    RETURN
ProductService.addProduct(request)
  END FUNCTION

  FUNCTION stockIn(productId, quantity)
    RETURN
ProductService.stockIn(productId, quantity)
  END FUNCTION

  FUNCTION stockOut(productId, quantity)
    RETURN
ProductService.stockOut(productId,
quantity)
  END FUNCTION
```

ProductService

```
CLASS ProductService
  FUNCTION addProduct(productData)
    validationResult =
ProductValidationService.validateNewProduct(p
roductData)
    IF validationResult != "VALID" THEN
      RETURN validationResult
    END IF
ProductRepository.save(productData)
    RETURN "Product added successfully"
  END FUNCTION
  FUNCTION stockIn(productId, quantity)
    product =
ProductRepository.findById(productId)
validationResult =
ProductValidationService.validateStockIn(produ
ct, quantity)
    IF validationResult != "VALID" THEN
      RETURN validationResult
    END IF
product.stockQuantity += quantity
ProductRepository.save(product)
TransactionService.log("STOCK_IN", productId,
quantity)
AlertService.checkLowStock(product)
    RETURN "Stock updated successfully"
  END FUNCTION
```

ProductRepository

```
CLASS ProductRepository
  FUNCTION findById(productId)
    RETURN product
  END FUNCTION
  FUNCTION findBySku(sku)
    RETURN product
  END FUNCTION
  FUNCTION save(product)
    store product
  END FUNCTION
END CLASS
```

Pseudocode for Each Class

ProductValidationService

```
CLASS ProductValidationService
  FUNCTION validateNewProduct(product)
    IF product.sku is empty THEN
      RETURN "SKU is mandatory"
    END IF
    IF ProductRepository.findBySku(product.sku) exists
THEN
      RETURN "Duplicate SKU"
    END IF
    IF product.unitPrice <= 0 THEN
      RETURN "Invalid price"
    END IF
    IF product.minStockLevel < 0 THEN
      RETURN "Invalid minimum stock level"
    END IF
    RETURN "VALID"
  END FUNCTION



    END IF
    RETURN "VALID"
  END FUNCTION
END CLASS
```

```
FUNCTION validateStockIn(product, quantity)
IF product does not exist THEN
RETURN "Product not found"
END IF
IF product.status != "ACTIVE" THEN
RETURN "Inactive product"
END IF
IF quantity <= 0 THEN
RETURN "Invalid quantity"
END IF
RETURN "VALID"
END FUNCTION
```

```
FUNCTION validateStockOut(product, quantity)
IF product does not exist THEN
RETURN "Product not found"
END IF
IF product.status != "ACTIVE" THEN
RETURN "Inactive product"
END IF
IF quantity <= 0 THEN
RETURN "Invalid quantity"
END IF
IF product.stockQuantity < quantity THEN
RETURN "Insufficient stock"
```

Workflow – Add Product

Flow:
UI → ProductController → ProductService
→ ProductValidationService
→ ProductRepository.save()
→ Response to UI

Workflow – Stock In

Flow:
UI → Controller → Service
→ Validate
→ Increase stock
→ Save
→ Log Transaction
→ Check Low Stock
→ Success Response
Stock Updated Successfully

Workflow – Stock Out

Flow:
UI → Controller → Service
→ Validate
→ Check sufficient stock
→ Reduce stock
→ Save
→ Log Transaction
→ Check Low Stock
→ Success Response

Advantages of Module 2

- Prevents duplicate SKUs
- Avoids negative stock
- Enforces business validation
- Maintains transaction logs
- Real-time stock updates
- Improves inventory reliability
- Fully integrated with Authentication Module

Conclusion

The Product / Inventory Management Module:

- Automates product operations
- Ensures data accuracy
- Implements strict validation rules
- Maintains stock integrity
- Logs every transaction
- Generates low stock alerts
- This module forms the core operational engine of Inventra.

References

- Java 17 Documentation
- Spring Boot Official Documentation
- Spring Security Documentation
- JWT Documentation
- MySQL Official Docs
- Hibernate / JPA Docs

The End